AssetTrust v0.51

Reproducible builds, dependency provenance and guarded database validation

RELEASE DECISION: NO-GO. This is an offline-tested engineering checkpoint, not an approval to deploy with real customer data.

What was implemented

- An npm supply-chain guard rejects direct Git, filesystem, insecure package references, untrusted tarball hosts and inconsistent dependency locks.
- A supervised runner provides safe read-only planning, lockfile preflight, build verification, disposable database contract execution and combined validation.
- Database writes require explicit approval, a local PostgreSQL test target and a confirmation matching the disposable database name.
- Failure evidence records step results and durations without copying connection strings, credentials or child console output.
- CI checks the supply-chain policy before npm ci and includes the new v0.51 tests. All 25 existing SQL migrations remain unchanged.

Verification conducted in this environment

Control	Observed result
Version suites v0.03-v0.51	49 / 49 PASSED
New v0.51 security tests	17 / 17 PASSED
Architecture, syntax, CI coverage, offline smoke	PASSED
SQL migration SHA-256 fingerprints	25 / 25 MATCHED
Release plan (read-only)	PASSED

Scope warning: the 17 new tests include simulated runner success/failure using temporary fixtures. They do not prove live npm installation, database migrations, or application behavior.

Outstanding release blockers

Authentic dependency lockfile

The source archive lacks package-lock.json. npm registry DNS resolution failed here. npm ci --offline refused installation; live npm audit was not run.

Compiled application

npm test and npm run build fail because NestJS, pg and Node.js type packages are not installed. No runtime or browser behavior was proven.

Real PostgreSQL evidence

No disposable PostgreSQL service was available. The existing RLS contracts were not executed; the unapproved test run was blocked before connecting.

Production infrastructure and Git history

No deployed reverse proxy, SSH/firewall configuration, backup/restore drill or full Git history was available. Secrets scans and real recipient/admin flows remain outstanding.

Operator sequence for connected Kali Linux

- Work on a clean branch of the canonical repository and preserve .git history, encryption keys, signing keys and tested backups.
- Generate a genuine package-lock.json from the trusted registry; review the dependency graph and run npm run supply-chain:v51.
- Run npm run release:validate:v51 to exercise npm ci, npm audit, all 49 version suites, full builds, API tests and Git-history scanning.
- Create a fresh isolated local assettrust_ci PostgreSQL/PostGIS database. Explicitly authorize it, then run npm run release:database:v51.
- Conduct actual two-API, multi-organization, MFA, recipient-sharing, browser, TLS, firewall, malware-upload and independent security tests before signoff.

Read-only release preflight: 3 of 11 prerequisites present. All source RLS progress remains unverified against a live database. Database-wide RLS, user memberships and remaining confidential tables need further work.

Final determination: HOLD PRODUCTION RELEASE. Green offline checks are not evidence that a networked application is safe or that official land ownership has been verified.

Detailed evidence: TEST_EVIDENCE_V51.md | Operator runbook: V51_KALI_VALIDATION.md | Handover: V51_HANDOVER.md